

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Курсовой проект
по курсу «Технологии параллельного программирования»

Обратная трассировка лучей (Ray Tracing) на GPU

Выполнил: В.М. Филиппов

Группа: М8О-410Б-22

Преподаватель: А.Ю. Морозов

Москва, 2025

Условие

Описание задачи:

1. Использование GPU для создание фотореалистической визуализации.
Рендеринг полужеркальных и полупрозрачных правильных геометрических тел.
Получение эффекта бесконечности. Создание анимации.
2. Гексаэдр, Октаэдр, Додекаэдр

Программное и аппаратное обеспечение

GPU.

- Compute Capability: 8.9
- Графическая память (Global memory): 8 GB GDDR6 (8188 MiB — видно в nvidia-smi)
- Shared memory per SM (мультипроцессор): до 100 KB (может конфигурироваться: 64 KB shared + 32 KB L1 или наоборот)
- Shared memory per block: до 48 KB
- Constant memory: 64 KB (аппаратно выделено)
- Регистры на потоковый мультипроцессор (SM): 65,536
- Максимум регистров на блок: 32K (зависит от конфигурации)
- Максимальное число потоков на блок: 1024
- Максимальное число блоков в сетке (grid): $2^{31} - 1$ по X, $2^{16} - 1$ по Y и Z
- Количество мультипроцессоров (SM): 24 (у RTX 4060)
- Макс. потоков на SM: 2048
- Итого потоков на GPU: $24 \times 2048 = 49,152$ одновременно резидентных

CPU.

- Архитектура. x86_64
- Модель. AMD Ryzen 5 3600 6-Core Processor
- Количество физических ядер. 6
- Потоков. 12 (2 потока на ядро)
- Частота (BogoMIPS). 7200.05
- Кэш L1 (данные/инструкции). 192 KiB / 192 KiB
- Кэш L2. 3 MiB
- Кэш L3. 16 MiB

RAM.

- Общий объём. 15 GiB
- Swap. 4 GiB (свободен)
- SSD. • Объём. 1 TB
- Использовано. 156 GB

ПО.

- OS. Windows 11 + WSL 2.0.
- IDE. Visual studio code.
- NVCC. 11.5 (Build V11.5.119)

Метод решения.

В рамках написания курсовой работы я написал параллельный алгоритм трассировки лучей Path Tracer, реализовал несколько типов материалов (локальных моделей освещения): ламбертовский диффузный материал, модель Фонга для металлов и формула Шлика (для вероятности отражения) с моделью поглощения внутри по закону Ламберта-Бера; кроме того, я реализовал BVH для ускорения рендеринга.

Для каждого пикселя на изображении запускается ядро. Для устранения эффекта алиасинга («лесенок») используется метод стохастической выборки: лучи запускаются не через центр пикселя, а со случайным смещением в пределах его площади. Далее луч последовательно пересекается с объектами, отражается/преломляется, пока не достигнет источника света, области неба или не будет остановлен по глубине. При каждом отскоке вектор пропускной способности (throughput) умножается на альбедо материала. Итоговый цвет сэмпла формируется как сумма эмиссионных компонентов, взвешенных на накопленный throughput

Пересечение луча и объектов на сцене для оптимизации считается через BVH. Вместо того, чтобы проверять пересечение луча сразу со всеми плоскостями или сферами, мы проверяем сначала крупные AABV боксы, и только если луч пересекает бокс, проверяем примитивы внутри.

Пересечение луча со сферой определяется через решение уравнения сферы. Проверяем, существуют ли корни у этого уравнения (то есть пересекает ли луч сферу), а далее находим ближайший корень, проверяя находится ли он в указанном интервале. Это необходимо для того, чтобы не проверять самопересечения и отсекал дальние объекты в случае BVH.

Пересечение луча и плоскости проверяется через решение уравнения для параметрического представления луча и плоскости. Первоначально проверяется, не параллелен ли луч плоскости ($\mathbf{n} \cdot \mathbf{d} \neq 0$).

$$t = \frac{\mathbf{n} \cdot (\mathbf{p}_0 - \mathbf{o})}{\mathbf{n} \cdot \mathbf{d}}$$

После нахождения точки пересечения мы переводим вектор в локальную двухмерную систему координат. Далее для каждого типа плоскости (квадрат, треугольник, круг) проверяется, лежит ли точка пересечения с бесконечной плоскостью внутри заданной фигуры.

Далее, если луч пересекает объект на сцене, мы начинаем вычислять цвет пикселя в зависимости от материала и текстуры. Сначала загружаем текстуру, умножая ее на изначальный цвет материала. Затем добавляем собственное свечение у материала. Далее для луча выбираем его дальнейшее направление и цвет в зависимости от материала.

Для ламбертовского материала (матовой поверхности) мы выбираем случайный луч в полусфере вокруг нормали, а в качестве возвращённого цвета используется albedo.

Для металлов вычисляется отраженный луч, а далее к этому отражённому лучу добавляется случайный луч в единичной сфере, который умножается на коэффициент шероховатости, чтобы симулировать неровность на металле. В качестве цвета используется albedo.

Для диэлектриков вычисляем сначала вычисляем коэффициент преломления из закона Снеллиуса, который зависит от того, входит ли луч в объект или выходит из него.

$$n_1 \sin \theta_1 = n_2 \sin \theta_2$$

Когда свет идёт из более плотной среды в менее плотную ($n_1 > n_2$) под достаточно большим углом, преломление невозможно — весь свет отражается.

Далее мы аппроксимируем уравнение Френеля при помощи формулы Шлика.

$$R(\theta) \approx R_0 + (1 - R_0)(1 - \cos \theta)^5, \text{ где } R_0 = \left(\frac{n_1 - n_2}{n_1 + n_2} \right)^2.$$

С помощью этой формулы мы можем узнать вероятность отражения света и в зависимости от этого вернуть луч соответствующего направления.

Свет проходя через диэлектрик всё равно частично может поглощаться им. Этот процесс симулируется при помощи модели Бера-Ламберта и описывается формулой.

$$I = I_0 e^{-\alpha x},$$

где I - интенсивность света после прохождения пути x ,

I_0 - начальная интенсивность,

α - коэффициент поглощения.

Поглощение применяется только при выходе из материала, потому что x — это расстояние, которое луч прошёл внутри стекла от входа до выхода

Описание программы

Описание архитектуры программы. Разделение по файлам, описание основных типов данных и функций. Обязательно описать реализованные ядра. Обосновать используемые решения (почему реализовали так, а не по другому).

Точка входа в программу находится в файле main.cu. Там же парсится конфигурация из файла, загружаются текстуры, реализуются функции создания специфических фигур (куб, октаэдр, додекаэдр). В файле camera.h описаны интерфейсы различных экспортёров (в бинарный формат, в ppm и png), а также класс камеры, ее описание, ядро

рендера и некоторые дополнительные функции. Часть реализаций вынесена в camera.cu. Кроме того, там также присутствует основная функция рендера для CPU алгоритма. В файле bvh.h реализован BVH, для геометрии и хит-функций есть файлы plane.h, sphere.h, aabb.h. В файле scene.h и scene_params.h описано взаимодействие со сценой. Также присутствуют вспомогательные классы в файлах vec3.h, ray.h, interval.h, random_utils.h.

CUDA-ядром рендера является render_kernel. Оно распараллеливает изображение по пикселям обрабатывая изображение блоками NxN. Были использованы константные буферы: __constant__ CameraData d_cam_data_const и __constant__ SceneData d_scene_data_const для быстрого чтения неизменяемых параметров. Для каждого пикселя ядро генерирует сэмплы, вызывает device-функцию трассировки ray_color аккумулялирует цвет и записывает в буфер.

В CUDA реализации отсутствует рекурсия для основных функций (BVH поиск, ray_color). У GPU очень небольшой стэк вызовов, поэтому использование рекурсий крайне нежелательно. Кроме того, рекурсия требует сохранения состояния регистров в локальную память (local memory), что значительно снижает производительность и увеличивает latency. Поэтому там, где в CPU мы могли бы спокойно использовать рекурсию, где она была бы удобна и к месту, использовалась итеративная замена.

Для ускорения рендеринга я использовал BVH, поскольку проверка на пересечение сразу со всеми объектами сцены – это очень ресурсоёмкий процесс. Особенно это заметно на сцене, которая является главным примером: там очень много сцен на ребрах фигур, сами эти ребра являются плоскостями, каждая из фигур – это набор плоскостей.

Архитектурно я отказался от использования виртуальных методов как в C++. Они сильно замедляли работу программы, поскольку обращение к таблице виртуальных методов вызывает непредсказуемые переходы в памяти инструкций и расхождение варпов. Кроме того, почти все функции были помечены как inline, что также сильно повлияло на производительность: функции подставляются компилятором напрямую в код, что увеличивает скорость рендера.

Сильно повлияло на производительность (почти двухкратное ускорение) использование вместо double float. На качество рендера точность вычисления влияет слабо, однако на передачу данных между функциями – существенно.

Также вместо обычных математических функций я использовал специальные CUDA аналоги с постфиксом -f (sqrtf, maxf, powf и другие).

Многие функции могут быть использованы как для GPU реализации трейсера, так и для CPU версии. Поэтому многие из них помечены как __host__ и __device__ одновременно. Это помогло избежать многократного дублирования кода.

Поскольку метод трассировки путей является стохастическим алгоритмом, основанным на методе Монте-Карло, качество изображения напрямую зависит от качества генератора псевдослучайных чисел (ГПСЧ). Для реализации на GPU был выбран алгоритм хеширования Томаса Ванга (Wang Hash). Данная функция выполняет серию битовых операций (XOR, сдвиги, умножения) над входным значением.

Исследовательская часть и результаты

Пока работал над тем, чтобы добиться получения хорошего эффекта зеркальных отражений, получился вот такой красивый додэкаэдр.

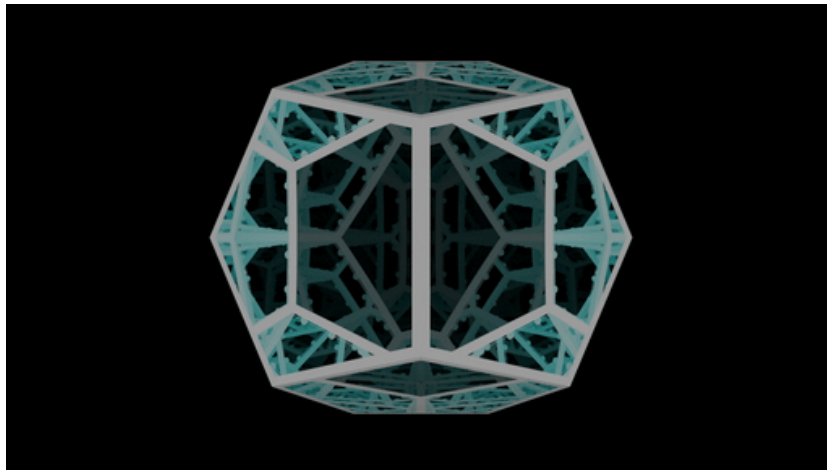


Рисунок 1. Додэкаэдр.

Также я замерил скорость выполнения в зависимости от количества нитей в блоке. Синее плато означает то, что с таким количеством потоков ядро запустить невозможно. Видим явный рост времени выполнения при уменьшении общего количества потоков.

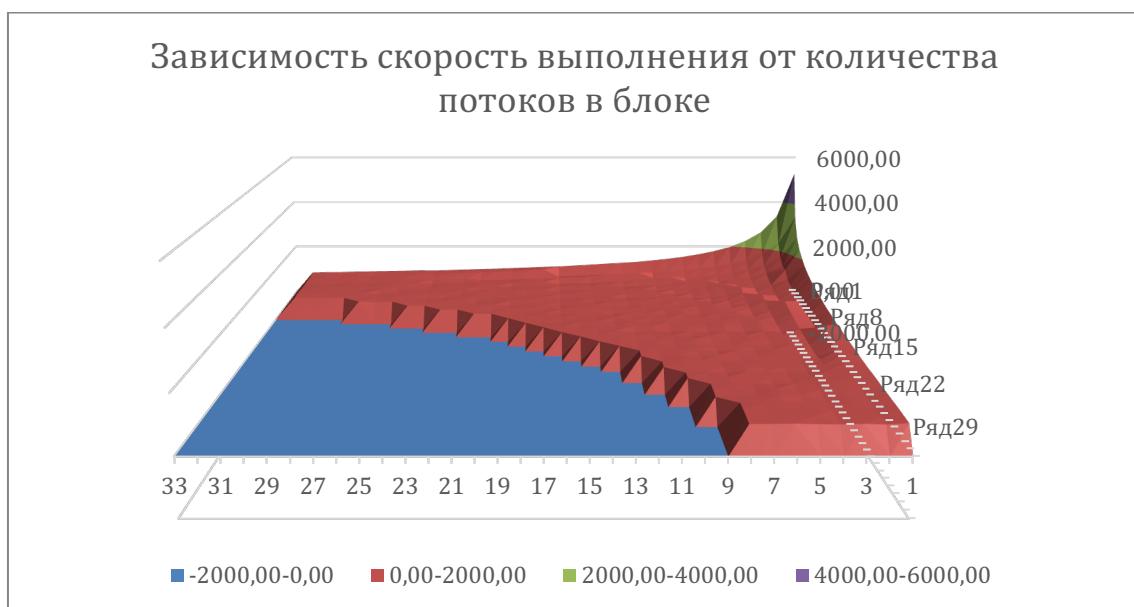


Рисунок 2. График зависимости скорости выполнения от количества потоков в блоке.

Кроме того, я экспортировал получившиеся полигоны на сцене и построил при помощи gnuplot сцену и траекторию движения камеры.

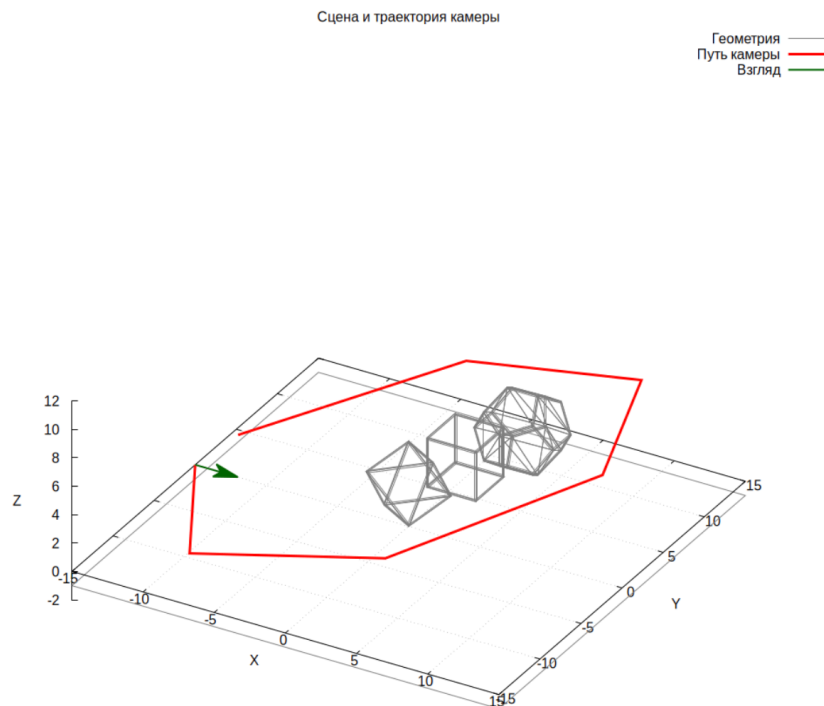


Рисунок 3. Траектория движения камеры вокруг объектов.

Я построил зависимость скорости исполнения от корня количества сэмплов для обработки. Видим параболу, как и ожидалось.



Рисунок 4. График зависимости времени исполнения от количества сэмплов

При небольшом количестве ресэмплирования изображение получается с эффектом акнэ, и к тому же очень темным. Если переборщить с ресэмплированием, то можем получить пересвет, который не очень хотелось бы видеть.

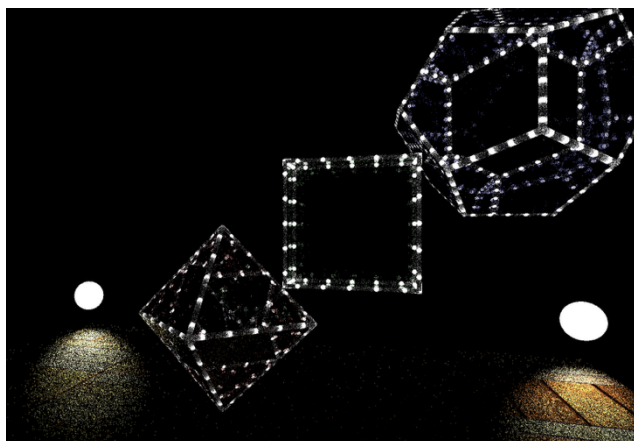


Рисунок 5. Изображение при $SPP = 1$

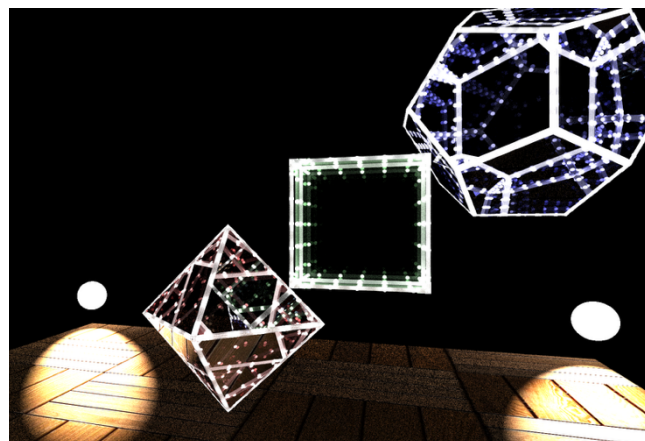


Рисунок 6. Изображение при $SPP = 50$

На изображении справа лучше видно эффект бесконечного отражения, поскольку пиксели окрашиваются с большей вероятностью при большем количестве сэмплов.

Я произвёл замеры скорости выполнения в зависимости от максимальной глубины рекурсии. На графике видно, что сначала график растёт довольно быстро, а потом выходит на плато. Это связано с тем, что рекурсия при рендеринге прозрачных объектов ограничивается методом русской рулетки. Его суть заключается в том, что если вклад луча становится слишком маленьким, рекурсия заканчивается.



Рисунок 7. График зависимости скорости выполнения от максимальной глубины рекурсии.



Рисунок 8. Изображение при $\text{max depth} = 1$

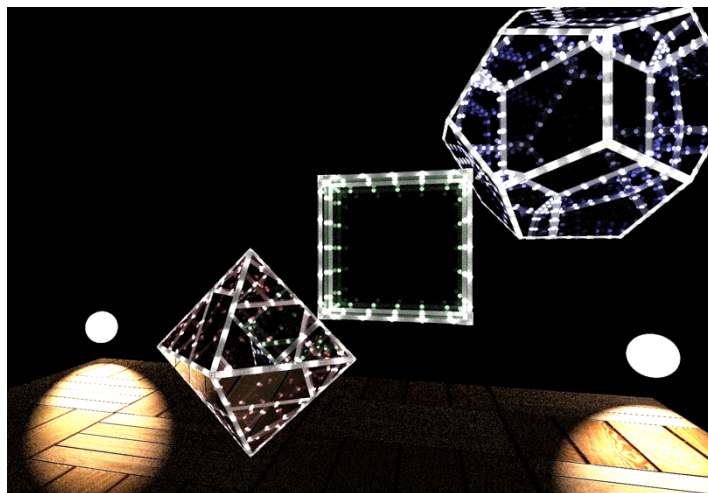


Рисунок 9. Изображение при $\text{max depth} = 20$

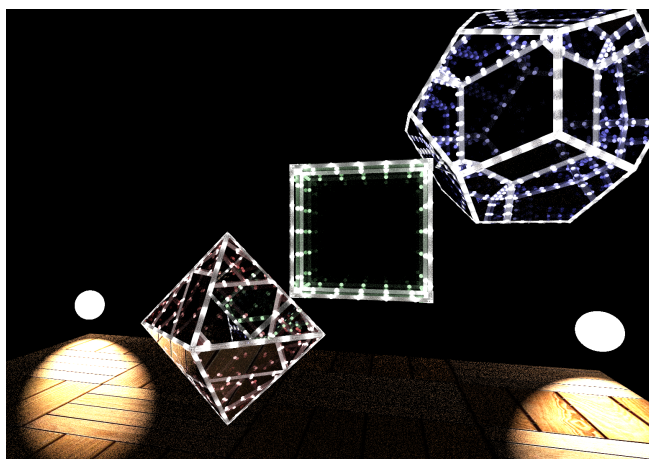


Рисунок 10. Изображение при $\text{max depth} = 40$

Мы видим, что при ограничении глубины рекурсии одной итерацией на изображении перестаёт быть что-либо видно. При глубине 20 изображение принимает изначальный вид, при этом оно мало чем отличается от изображения при глубине 40. Этот эффект мы уже обсуждали выше.

На рисунке 11 изображен график зависимости времени исполнения от номера кадра. На нём мы видим явный скачок времени исполнения в середине анимации. При этом до и после видны некоторые просадки по времени исполнения. На кадре с самым длительным временем исполнения мы почти во весь экран видим додэкаэдр, из-за которого время исполнения и увеличивается из-за большого количества отскоков луча внутри него, а также отражении пола. На кадре с наименьшим временем исполнения фигуры удалены от нас, поэтому количество лучей, которые попадают по фигурам минимально, что и даёт нам увеличение скорости рендеринга.



Рисунок 11. График зависимости времени исполнения от номера кадра

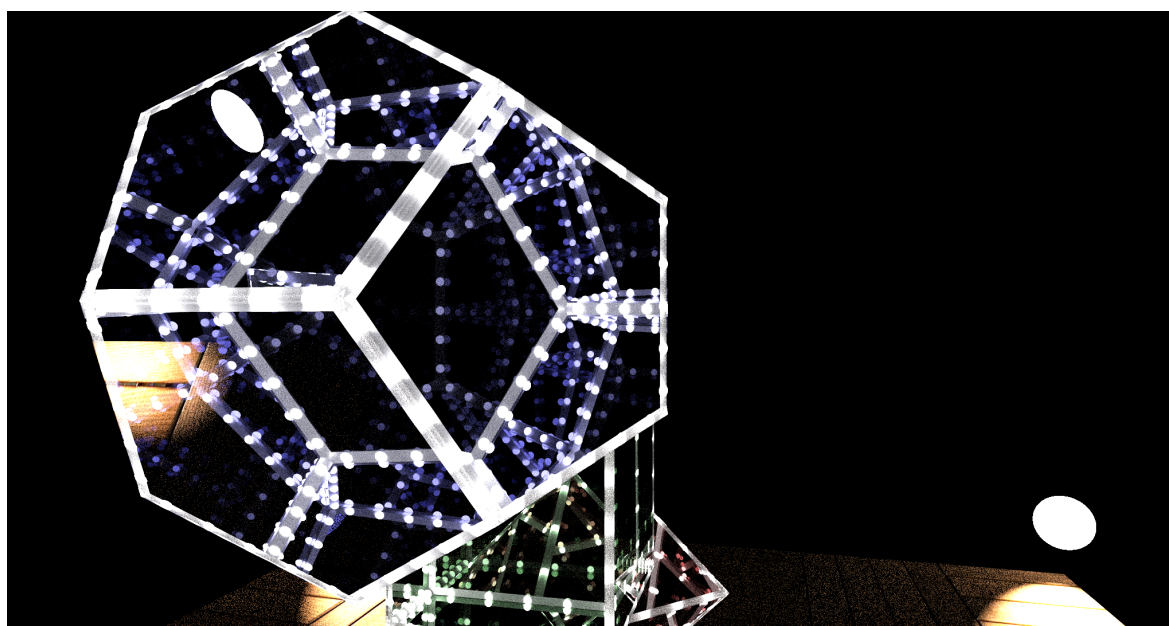


Рисунок 12. Кадр с самым длительным временем рендеринга.

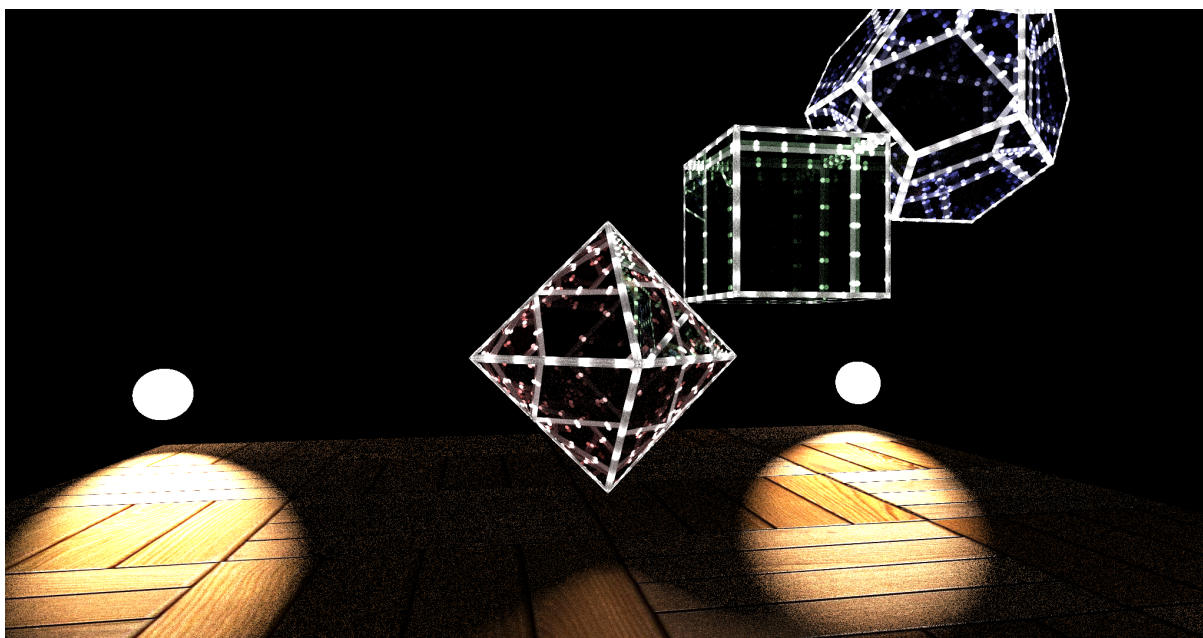


Рисунок 13. Кадр с наименьшим временем рендеринга.

Если убрать источники света и сделать пол более отражающим, то мы сможем достигнуть вот такого эффекта зеркала на полу.

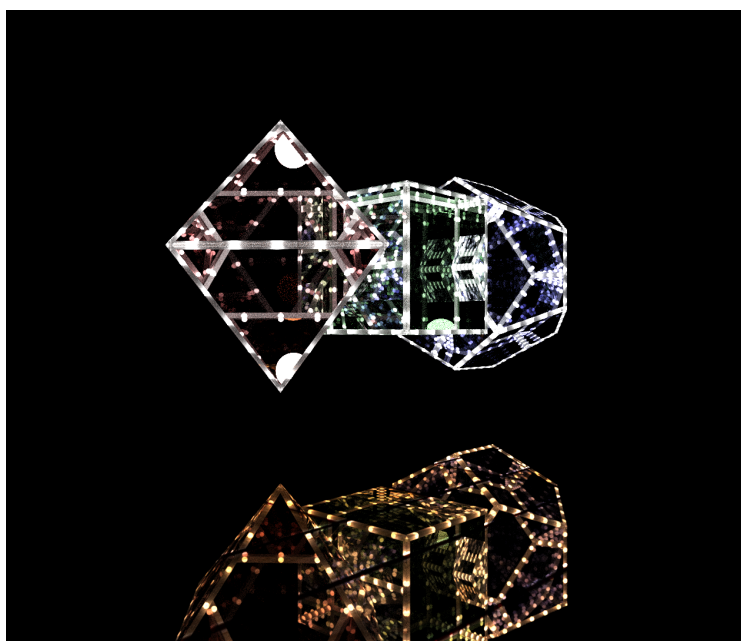


Рисунок 14. Пол с зеркальным эффектом.

Дополнительно я произвёл замеры скорости исполнения GPU и CUDA версии. График времени исполнения относительно номера кадра для GPU и CPU представлен на рисунке 15. На рисунке 16 этот же график, но в логарифмической шкале. Скорость исполнения на CPU при одинаковых конфигурациях сцены примерно в 75-80 раз медленнее, чем на GPU.

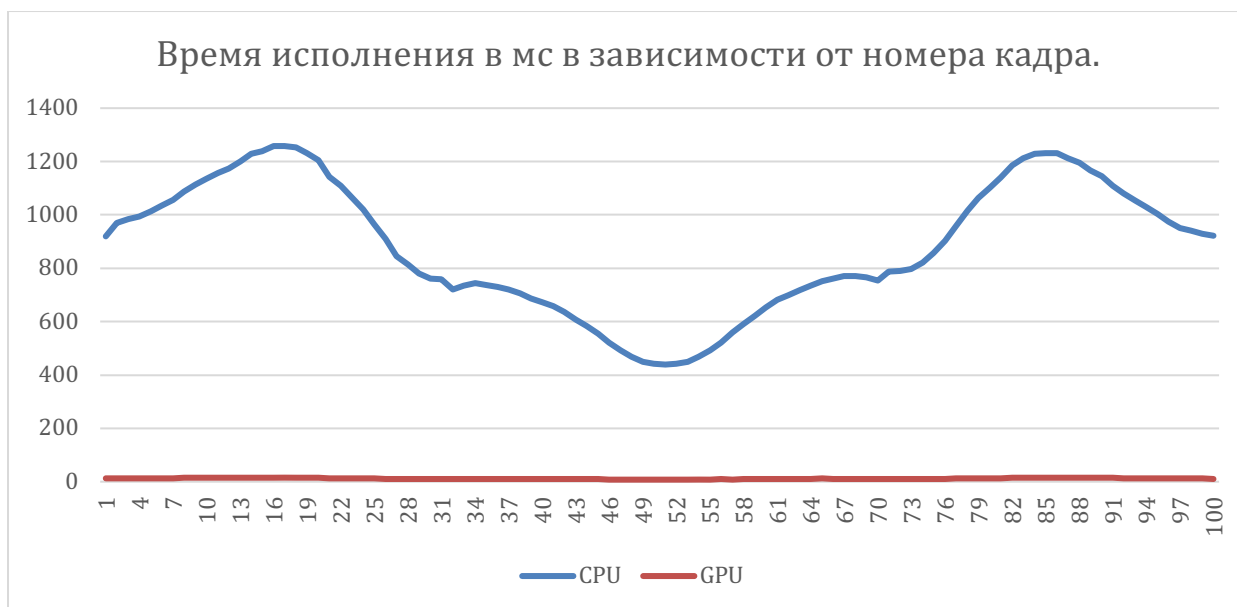


Рисунок 15. Графики зависимости времени исполнения от номера кадра

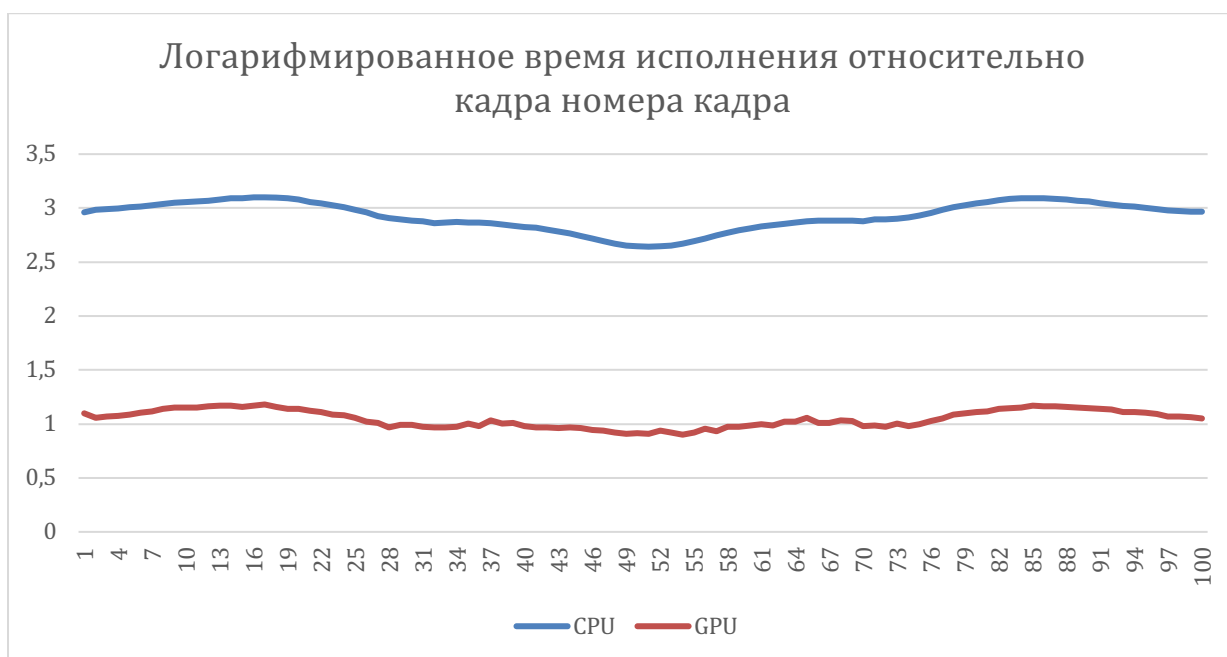


Рисунок 16. Графики зависимости времени исполнения от номера кадра в логарифмической шкале

Посмотрим, отличаются ли визуально изображение, которое отрисованного на CPU от изображения отрисованного на GPU. По рисункам 17 и 18 видно, что отличия минимальны. Во многом даже эта разница объясняется тем, что в CPU реализации используется другая функция для получения случайного числа, что определенно влияет на результат.

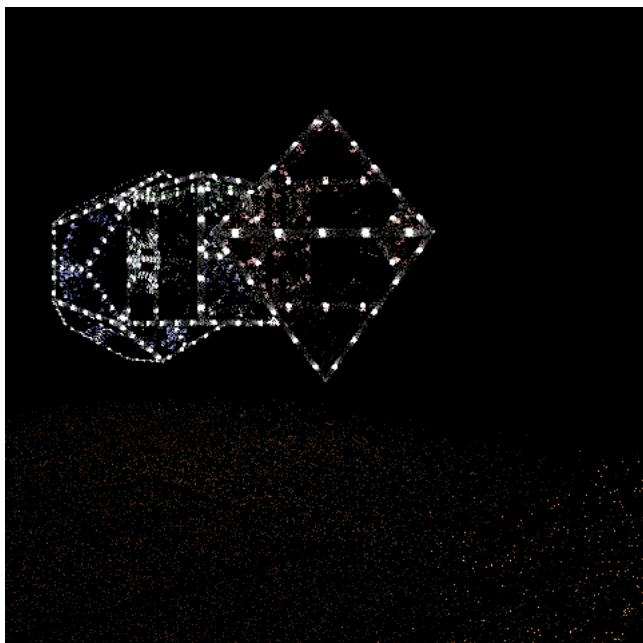


Рисунок 17. Кадр, отрисованный на GPU

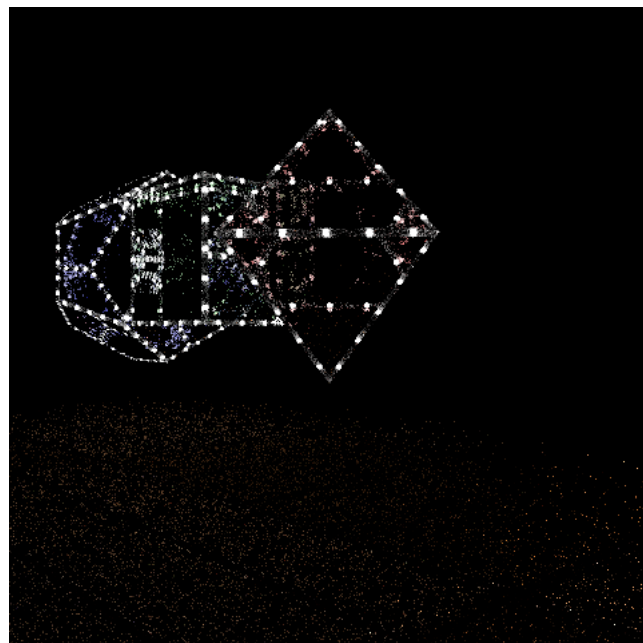


Рисунок 18. Кадр, отрисованный на CPU

И напоследок изображение, которое я рендерил, пока тестировал наложение текстур.
Крутой лев!

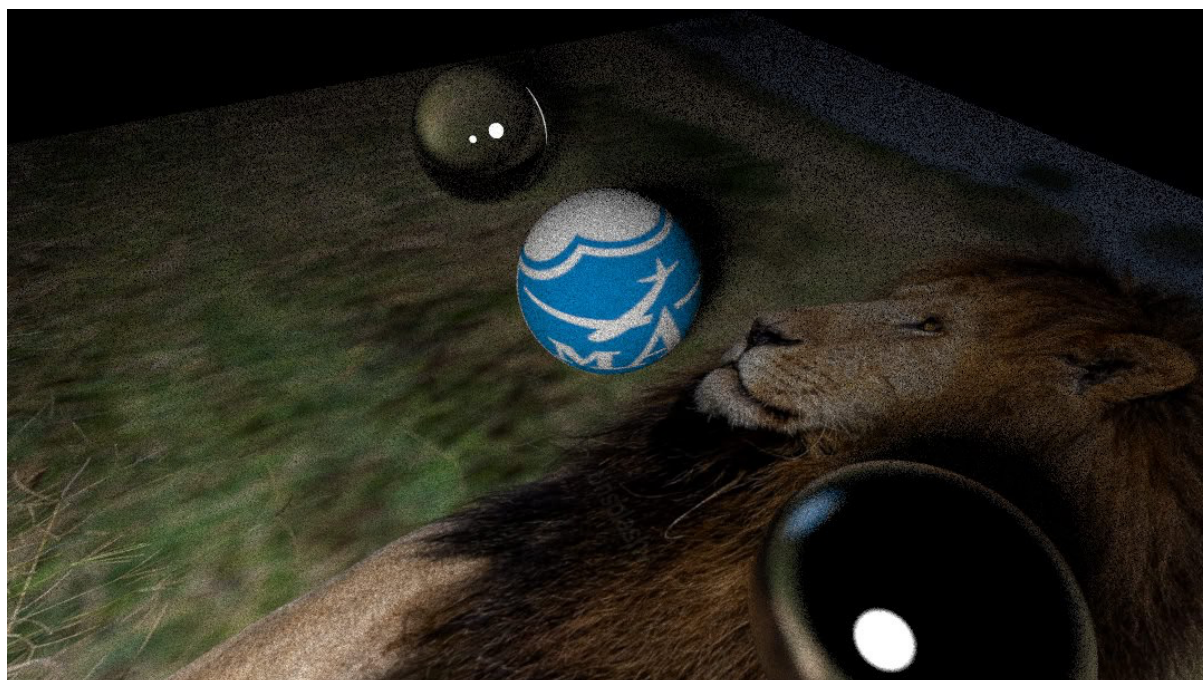


Рисунок 17. Наложение текстур на сферы и плоскости.

Выводы

Разработанный алгоритм трассировки лучей на GPU применяется в задачах фотореалистичной визуализации, где требуется моделирование сложных эффектов освещения: отражения, преломления, глобального освещения. Он используется в компьютерной графике для создания изображений и анимаций с высоким качеством, в архитектурной визуализации, дизайне, киноиндустрии, а также в научных симуляциях, например, для моделирования оптических систем.

Программирование этого алгоритма – это довольно трудоёмкий процесс, пришлось вспоминать некоторые знания из линейной алгебры, физики и геометрии. Конечно, пригодились знания полученные на курсе «Программирование графических процессоров», без которых я едва ли смог написать CUDA версию алгоритма обратной трассировки лучей.

Проблемы во время написания алгоритма возникали постоянно. Самым скучным и мучительным было правильно посчитать и выставить в нужном порядке вершины для построения фигур из плоскостей так, чтобы нормали указывали в нужном направлении. Долго я мучился с тем, чтобы достичь эффекта «бесконечного зеркала», я никак не мог подобрать правильные параметры для этого. Кроме того, в процессе написания кода постоянно возникали ситуации, когда вместо изображения я получал пустое полотно.

Тесты показали, что GPU-реализация работает примерно в 75–80 раз быстрее, чем CPU-версия при одинаковых сценах. Это объясняется высокой степенью параллелизма и эффективным использованием ресурсов GPU. При увеличении глубины рекурсии качество изображения растёт, но после определённого уровня (20–40) изменения становятся минимальными, что связано с применением метода «русской рулетки» для ограничения трассировки. Также наблюдается зависимость времени рендеринга от сложности сцены и количества сэмплов: при большом числе отражающих поверхностей время растёт из-за увеличения числа пересечений лучей.

Литература

1. Ray Tracing in One Weekend // Ray Tracing in One Weekend — The Book Series URL: <https://raytracing.github.io/books/RayTracingInOneWeekend.html> (дата обращения: 26.11.2025).
2. Ray Tracing: The Next Week // Ray Tracing in One Weekend — The Book Series URL: <https://raytracing.github.io/books/RayTracingTheNextWeek.html> (дата обращения: 29.11.2025).
3. Ray Tracing: The Rest of Your Life // Ray Tracing in One Weekend — The Book Series URL: <https://raytracing.github.io/books/RayTracingTheRestOfYourLife.html> (дата обращения: 30.11.2025).
4. Accelerated Ray Tracing in One Weekend in CUDA // Developer Nvidia URL: <https://developer.nvidia.com/blog/accelerated-ray-tracing-cuda/> (дата обращения: 01.12.2025).